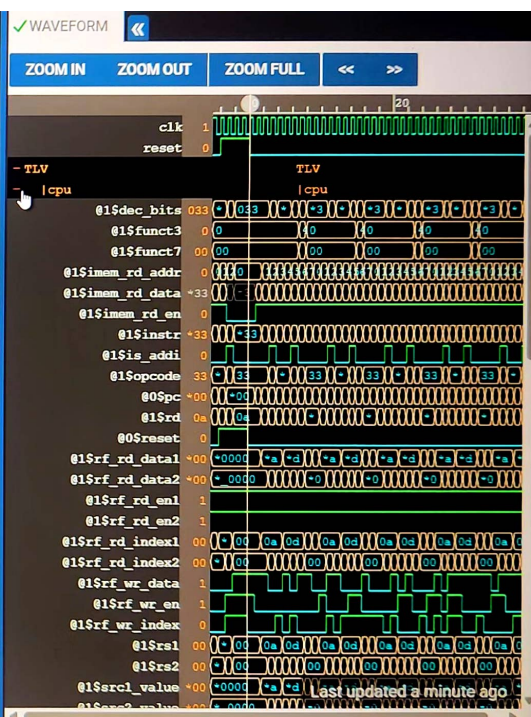
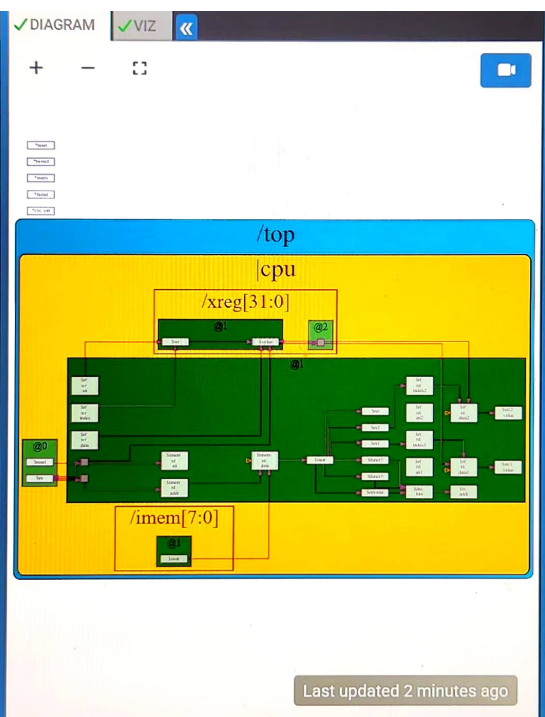


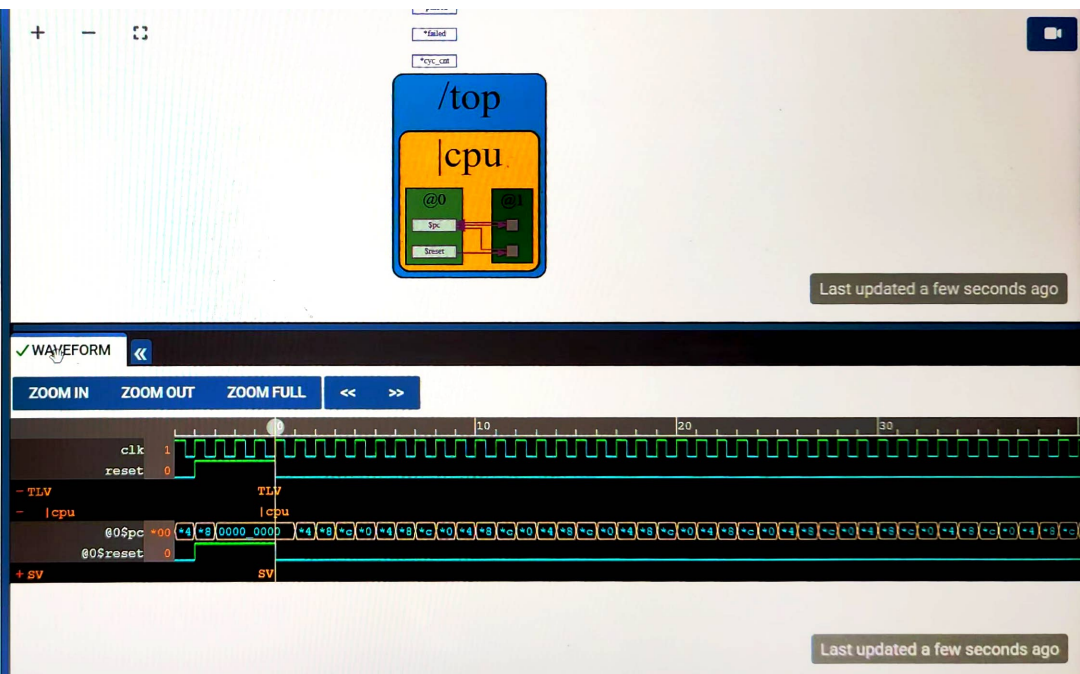
```
44 >>1$pc + 32'd4;
45
46 @1
47 $imem_rd_en = !$reset;
48
49 $imem_rd_addr[M4_IMEM_INDEX_CNT-1:0] = {
50
51 $instr[31:0] = $imem_rd_data;
52 $opcode[6:0] = $instr[6:0];
53
54 $rs1[4:0] = $instr[19:15];
55 $rs2[4:0] = $instr[24:20];
56 $rd[4:0] = $instr[11:7];
57
58 $funct3[2:0] = $instr[14:12];
59 $funct7[6:0] = $instr[31:25];
60
61 $dec_bits[10:0] = {$funct7[5], $funct3,
62
63 $is_addi = $dec_bits ==? 11'bx_000_00100
64
65 $rfd_en1 = 1'b1;
66 $rfd_en2 = 1'b1;
67
68 $rfd_index1[4:0] = $rs1;
69 $rfd_index2[4:0] = $rs2;
```




```

40 |cpu
41 @0
42 $reset = *reset;
43 $pc[31:0] = >>1$reset ? 32'd0 :
44 >>1$pc + 32'd4;
45
46
47
48 // YOUR CODE HERE
49 // ...
50
51 // Note: Because of the magic we are using
52 // be sure to avoid having unassigned
53 // other than those specifically expected
54
55
56 // Assert these to end simulation (before Main)
57 *passed = *cyc_cnt > 40;
58 *failed = 1'b0;
59
60 // Macro instantiations for:
61 // o instruction memory
62 // o register file
63 // o data memory
64 // o CPU visualization
65 |cpu

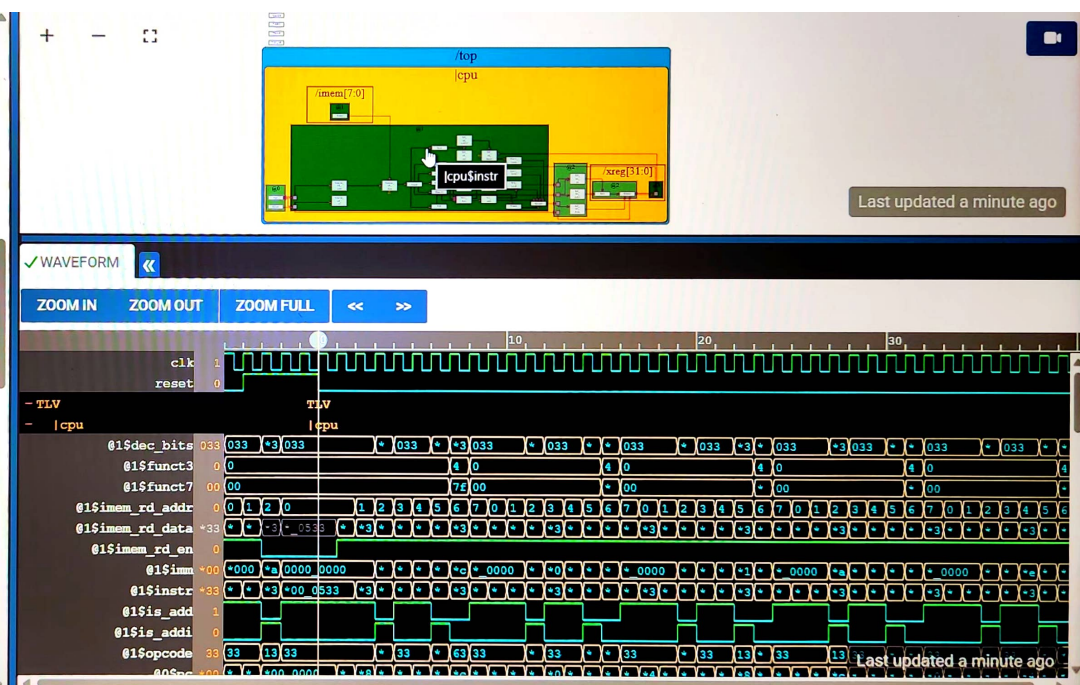
```



```

40 *|cpu
41 @0
42 $reset = *reset;
43 $pc[31:0] = >>1$reset ? 32'd0 :
44 >>1$pc + 32'd4;
45
46 @1
47 $mem_rd_en = !$reset;
48
49 $mem_rd_addr[M4_IMEM_INDEX_CNT-1:0] = $
50
51 $instr[31:0] = $mem_rd_data;
52 $opcode[6:0] = $instr[6:0];
53
54 $rs1[4:0] = $instr[19:15];
55 $rs2[4:0] = $instr[24:20];
56 $rd [4:0] = $instr[11:7];
57
58 $funct3[2:0] = $instr[14:12];
59 $funct7[6:0] = $instr[31:25];
60
61 $dec_bits[10:0] = {$funct7[5], $funct3,
62
63 $is_addi = $dec_bits ==? 11'bx_000_00100
64
65 $is_add = $dec_bits ==? 11'b0_000_011001
66

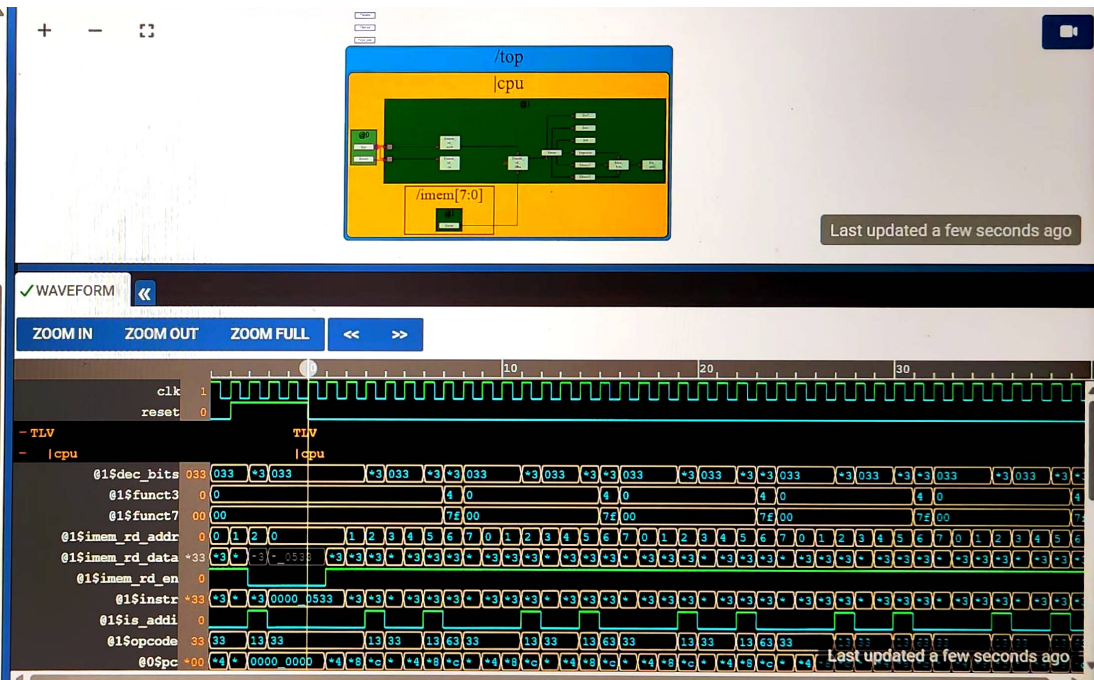
```




```

39 |cpu
40 @0
41 $reset = *reset;
42 $pc[31:0] = >>1$reset ? 32'd0 :
43 >>1$pc + 32'd4;
44
45
46 @1
47 $mem_rd_en = !$reset;
48
49 $mem_rd_addr[M4_IMEM_INDEX_CNT-1:0]
50
51 $instr[31:0] = $mem_rd_data;
52 $opcode[6:0] = $instr[6:0];
53
54 $rs1[4:0] = $instr[19:15];
55 $rs2[4:0] = $instr[24:20];
56 $rd[4:0] = $instr[11:7];
57
58 $funct3[2:0] = $instr[14:12];
59 $funct7[6:0] = $instr[31:25];
60
61 $dec_bits[10:0] = {$funct7[5], $funct3};
62
63 $is_addi = $dec_bits == 11'bx_000_
64
65

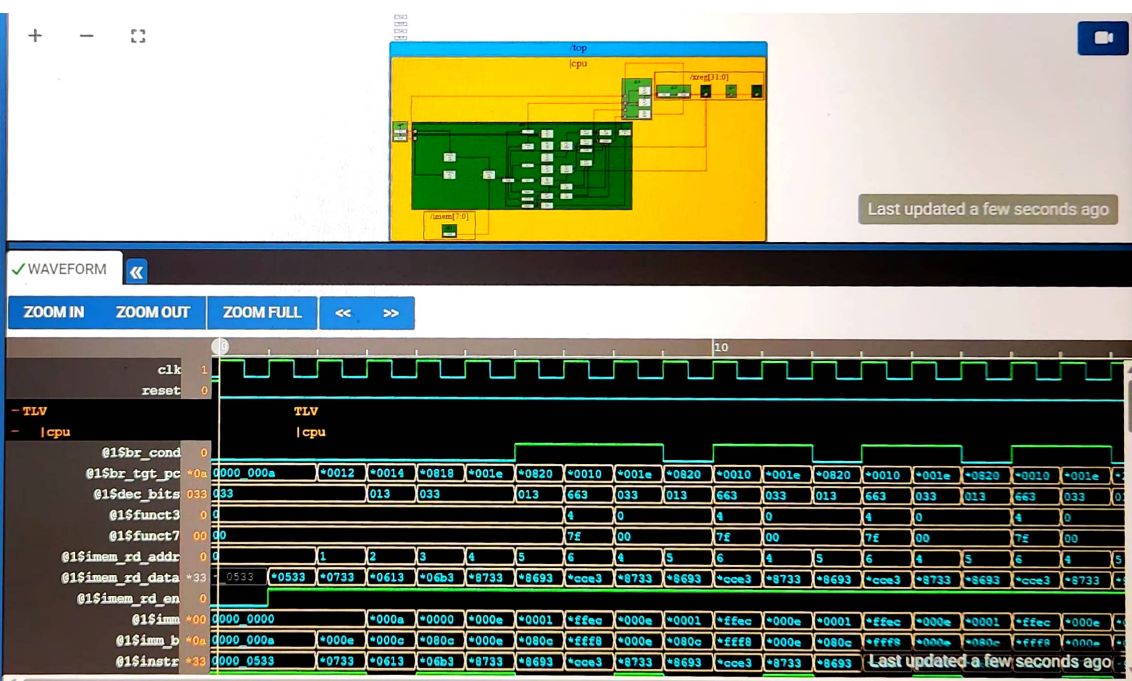
```



```

55 Srd[4:0] = $instr[11:7];
56
57 $func3[2:0] = $instr[14:12];
58
59 $func7[6:0] = $instr[31:25];
60
61 $dec_bits[10:0] = {$func7[5], $fur
62
63 $sis_addi = $dec_bits ==? 11'bx_000_
64
65 $sis_add = $dec_bits ==? 11'b0_000_
66
67 $rf_rd_en1 = 1'b1;
68 $rf_rd_en2 = 1'b1;
69
70 $rf_rd_index1[4:0] = $rs1;
71 $rf_rd_index2[4:0] = $rs2;
72
73 $src1_value[31:0] = $rf_rd_data1;
74 $src2_value[31:0] = $rf_rd_data2;
75
76 $imm[31:0] = {{21{$instr[31]}}, $sr
77
78 $imm_b[31:0] = {{19{$instr[31]}}, $
79
80 $result[31:0] = $sis_add ? ($src1\
81 $sis_addi ? ($src1\

```



```

41 2
42 $reset = *reset;
43 $pc[31:0] = >>1$reset ? 32'd0 :
44 >>1$pc + 32'd4;
45
46 7
47 $mem_rd_en = !$reset;
48
49 $mem_rd_addr[M4_IMEM_INDEX_CNT-1:0]
50
51 $instr[31:0] = $mem_rd_data;
52 $opcode[6:0] = $instr[6:0];
53
54 $rs1[4:0] = $instr[19:15];
55 $rs2[4:0] = $instr[24:20];
56 $rd[4:0] = $instr[11:7];
57
58 $funct3[2:0] = $instr[14:12];
59 $funct7[6:0] = $instr[31:25];
60
61 $dec_bits[10:0] = {$funct7[5], $funct
62
63 $is_addi = $dec_bits ==? 11'bx_000_00
64
65 $is_add = $dec_bits ==? 11'b0_000_011
66
67

```

